Perhaps it takes each developer three tries to get it right. Since running the test is faster than verifying in the browser, Jamie gains a little bit each try. After verifying the code three times, Jamie is only two and a half minutes behind Sam. (In a slight nod to reality, let's assume that both of them need to verify one last time in the browser once they think they are done. Because they both need to do this, it's not an advantage for either one.)

At this point, with the task complete, both break for lunch (a burrito for Jamie, an egg-salad sandwich for Sam—thanks for asking). After lunch they start on the next task, which is a special case of the first task. Jamie has most of the test setup in place, so writing the test takes only two minutes. Still, it's not looking good for Jamie, even after another three rounds trying to get the code right. Jamie remains a solid two minutes behind Sam.

Let's get to the punch line. Sam and Jamie are both conscientious programmers, and they want to clean up their code with a little *refactoring*, meaning that they are improving the code's structure without changing its behavior. Now Sam is in trouble. Each time Sam tries the refactoring, it takes two minutes to verify both tasks, but Jamie's test suite still takes only about 10 seconds. After three more tries to get the refactoring right, Jamie finishes the whole thing and checks it in three and a half minutes ahead of Sam. (Jamie then catches a train home and has a pleasant evening. Sam just misses the train and gets caught in a sudden rainstorm. If only Sam had run tests.)

My story is simplified, but look at all the things I didn't assume. I didn't assume that Jamie spent less actual time on tasks, and I didn't assume that the tests would help Jamie find errors more easily—although I think Jamie would, in fact, find errors more easily. (Of course, I also didn't assume that Jamie would have to track down a broken test in some other part of the application.)

It is frequently faster to run multiple verifications of your code as an automated test than to always check manually. And that advantage only increases as the code gets more complex. Additionally, the automated check will do a better job of ensuring that steps aren't forgotten.

There are many beneficial side effects of having accurate tests. You'll have better-designed code in which you'll have more confidence. But the most important benefit is that if you do testing well, your work will go faster. You may not see it at first, but at some point in a well-run, test-driven project, you'll notice that you have fewer bugs and that the bugs that do exist are easier to find. It will be easier to add new features and modify existing ones. You'll be doing better on the only code-quality metric that has any validity: